

DOKUMENTACJA TECHNICZNA

meczomat.pl

Dokumentacja projektu – wdrożenie nowego dewelopera

Wersja: 2.0 · Maj 2026 · Projekt: Meczomat v2.0

Spis treści

1.	Opis projektu i cel	3
2.	Architektura systemu – przegląd ogólny	4
3.	Backend – serwer Node.js (server.js)	5
4.	Baza danych – MongoDB Atlas	7
5.	Frontend – aplikacja React (meczomat2.0/)	8
6.	Omówienie plików frontendu	9
7.	API – lista wszystkich endpointów	11
8.	System CMS – panel redakcyjny	12
9.	Uruchomienie projektu lokalnie	13
10.	Deployment – produkcja	14
11.	Znane ograniczenia i TODO	15

1

Opis projektu i cel

Meczomat.pl to serwis internetowy poświęcony wynikom i tabeli ligowej polskich rozgrywek piłkarskich – ze szczególnym naciskiem na **niższe szczeble** (IV liga, V liga, Klasa Okręgowa, A-Klasa), które są pomijane przez popularne portale sportowe.

GŁÓWNA IDEA

Kibice małych, lokalnych klubów często nie mają dostępu do aktualnych wyników i tabeli swojej ligi w jednym miejscu. Meczomat.pl agreguje te dane automatycznie.

Funkcje dla użytkownika końcowego



Tabele ligowe

Aktualne tabele z Ekstraklasy aż po A-Klasę – z podziałem na województwa i grupy.



Terminarz meczów

Wyniki i harmonogram rozgrywek z podziałem na kolejki i daty.



Ulubiony klub

Użytkownik może zapisać swoją drużynę – serwis pokaże następny mecz i profil klubu.



Centrum Kibica

Artykuły, skróty wideo z YouTube oraz transmisje live dla fanów niższych lig.

Docelowa grupa użytkowników

- Kibice i zawodnicy drużyn grających w IV–A klasie rozgrywkowej
- Działacze i trenerzy lokalnych klubów
- Dziennikarze sportowi relacjonujący niższe ligi

2

Architektura systemu – przegląd ogólny

Projekt jest podzielony na dwa niezależne komponenty: **backend** (serwer API) i **frontend** (aplikacja React). Komunikują się przez REST API przez sieć.



Rysunek 1. Przepływ danych w systemie

Struktura katalogów projektu

ŚCIEŻKA	OPIS
<code>server.js</code>	Główny plik backendu – serwer Express, scraping, API
<code>package.json</code> (root)	Zależności backendu (express, mongoose, cheerio...)
<code>meczomat2.0/</code>	Cały frontend React + Vite + Tailwind CSS
<code>meczomat2.0/src/App.jsx</code>	Główny komponent React – nawigacja, widoki, kreator lig
<code>meczomat2.0/src/LeagueTable.jsx</code>	Komponent tabeli ligowej
<code>meczomat2.0/src/MatchList.jsx</code>	Komponent listy meczów / terminarza
<code>meczomat2.0/src/TeamSearch.jsx</code>	Wyszukiwarka i profil drużyny
<code>meczomat2.0/src/CmsPanel.jsx</code>	Panel redakcyjny (CMS) dla dziennikarzy
<code>meczomat2.0/src/PublicNews.jsx</code>	Centrum Kibica – publiczne artykuły i wideo

WAŻNE

Backend i frontend to dwa *osobne projekty Node.js* z osobnymi plikami `package.json`. Każdy uruchamia się niezależnie. Do produkcji backend jest hostowany na Render.com, frontend na Netlify/Vercel lub podobnym.

Technologie użyte w projekcie

WARSTWA	TECHNOLOGIA	WERSJA	ZASTOSOWANIE
Backend runtime	Node.js	—	Środowisko uruchomieniowe serwera

Backend framework	Express.js	^5.2	Routing HTTP i middleware
Web scraping	axios + cheerio	^1.15 / ^1.2	Pobieranie i parsowanie HTML z 90minut.pl
Kodowanie znaków	iconv-lite	^0.7	Konwersja ISO-8859-2 → UTF-8
Baza danych	MongoDB + Mongoose	^9.4	Przechowywanie artykułów, wideo, streamów
Frontend framework	React	^19.2	Interfejs użytkownika (SPA)
Build tool	Vite	^8.0	Bundlowanie i dev server frontendu
Stylowanie	Tailwind CSS	^4.2	Utility-first CSS

3

Backend – serwer Node.js (server.js)

Plik `server.js` to **jednorazowy monolit** – jeden plik obsługuje całe API. Uruchamia się jako serwer HTTP na porcie 3001 (domyślny) lub na porcie ze zmiennej środowiskowej `PORT`.

Słownik lig (LEAGUES)

Na samej górze pliku zdefiniowany jest słownik mapujący identyfikator ligi na URL strony 90minut.pl:

```
const LEAGUES = {
  'ekstraklasa': 'http://www.90minut.pl/liga/1/liga14072.html',
  '1-liga':      'http://www.90minut.pl/liga/1/liga14073.html',
  'iv-liga':     'http://www.90minut.pl/liga/1/liga14169.html',
  // ... inne ligi
  'a-klasa':     'TUTAJ_WKLEJ_LINK_Z_90MINUT' // ← do uzupełnienia!
};
```

JAK DODAC NOWĄ LIGĘ?

1. Wejdź na 90minut.pl, znajdź stronę ligi. 2. Skopiuj URL (musi zawierać `ligad+.html`). 3. Dodaj nowy klucz do obiektu `LEAGUES`. 4. Odpowiednio uzupełnij `LEAGUE_STRUCTURE` w `App.jsx`.

Mechanizm scrapingu – `fetchFrom90Minut()`

To najważniejsza funkcja w projekcie. Pobiera stronę HTML z 90minut.pl i parsuje ją:

1 Pobranie HTML

Używa `axios.get(url, { responseType: 'arraybuffer' })` – pobiera surowe bajty (strona używa kodowania ISO-8859-2, nie UTF-8).

2 Konwersja kodowania

`iconv.decode(response.data, 'iso-8859-2')` – zamienia polskie znaki zakodowane w starym standardzie.

3 Parsowanie tabeli

Cheerio (jQuery po stronie serwera) szuka elementu `table.main2` i odczytuje wiersze: pozycja, nazwa klubu, mecze, punkty, Z/R/P, bramki.

4 Parsowanie meczów

Szuka elementów `table.main`. Wyodrębnia kolejki (nagłówki „Kolejka X”) i wiersze z mecz: gospodarz, wynik, gość, data. Mecze bez wyniku (-) oznacza jako 'Nierozegrany'.

5 Zwraca obiekt

Funkcja zwraca `{ tabela: [...], mecze: [...] }` gotowy do oddania klientowi.

System cache'u

Dane z 90minut.pl są cache'owane w pamięci serwera na **15 minut**, aby nie przeciążać serwisu i przyspieszyć odpowiedzi:

ZMIENNA	TYP	OPIS
cache	Obiekt	Słownik <code>{ ligaId → { tabela, mecze, lastFetchTime } }</code>
fetchPromises	Obiekt	Deduplikacja równoległych requestów – jeśli ktoś pyta o tę samą ligę, czeka na już trwające pobieranie
CACHE_TIME	Number	15 minut (15 * 60 * 1000 ms)

WAŻNA UWAGA

Cache jest przechowywany *tylko w RAM* serwera. Po restarcie serwera cache jest pusty i pierwsze zapytanie każdej ligi spowoduje fetch z 90minut.pl (może trwać kilka sekund).

Lista redaktorów (JOURNALISTS)

Uprawnienia do CMS są hardcodowane w pliku serwera jako zwykły obiekt:

```
const JOURNALISTS = {
  'admin':    'haslo123',
  'redaktor': 'pilka2025',
  'kamera':   'wideo123'
};
```

⚠ Uwaga bezpieczeństwa! Hasła są przechowywane jako plain text. W przyszłości należy wdrożyć haszowanie (bcrypt) i JWT do autoryzacji. Tymczasowo rozwiązanie jest wystarczające dla małego zespołu redakcyjnego.

4

Baza danych – MongoDB Atlas

Baza danych przechowuje wyłącznie treści CMS (artykuły, wideo, transmisje). Wyniki sportowe **nie są zapisywane** w bazie – są pobierane na żywo ze scrapingu.

Połączenie

Serwer łączy się z chmurową bazą MongoDB Atlas przy starcie:

```
mongoose.connect('mongodb+srv://PanKoza:...@cluster0.ijaep7g.mongodb.net/')
```

⚠ **String połączenia jest na razie hardcoded w pliku!** Docelowo powinien być w zmiennej środowiskowej `.env`. Nigdy nie commituj hasła do repozytorium!

Modele danych (Mongoose Schemas)

KOLEKCJA	POLA	OPIS
articles	title, content, author, date	Artykuły redakcyjne, np. relacja z meczu. Pole <code>content</code> to długi tekst.
videos	title, embedUrl, author, date	Skróty wideo. <code>embedUrl</code> to przetworzony link YouTube embed (np. <code>youtube.com/embed/XXXX</code>).
streams	title, embedUrl, author, date	Transmisje na żywo. Identyczna struktura jak wideo.

Każdy dokument ma także autogenerowane pola `_id` i `createdAt` / `updatedAt` (dzięki opcji `{ timestamps: true }`).

Jak dodać nowe konto redaktora?

1

Otwórz plik `server.js`

Znajdź obiekt `JOURNALISTS` na początku pliku.

2

Dodaj nowy wpis

Dopisz linię: `'nowy_redaktor': 'silne_haslo'`

3

Zrestartuj serwer

Nowy redaktor może się zalogować w panelu CMS.

Frontend – aplikacja React (meczomat2.0/)

Frontend to **Single Page Application (SPA)** zbudowana w React 19 z Vite jako bundlerem i Tailwind CSS do stylowania. Wszystkie strony renderują się po stronie klienta (CSR).

Struktura nawigacji (widoki)

Aplikacja nie używa React Router – stan `currentView` w `App.jsx` decyduje o tym, co jest wyświetlane:

WARTOŚĆ CURRENTVIEW	CO WYŚWIETLA	KOMPONENT
'home'	Strona główna z widgetem ulubionej drużyny, nowościami i wyszukiwarką globalną	Wbudowane w App.jsx
'leagues'	Kreator wyboru ligi, tabela, mecze i profil drużyny	LeagueTable, MatchList, TeamSearch
'news'	Centrum Kibica – artykuły, wideo, transmisje	PublicNews
'cms'	Panel CMS dla redaktorów (wymaga logowania)	CmsPanel

Kreator wyboru ligi (Wizard)

Widok „Rozgrywki” to kreator krokowy prowadzący użytkownika przez wybór ligi. Sterowany zmienną `wizardStep`:

KROK	CO SIĘ DZIEJE
Krok 1	Wybór szczebla rozgrywek: Ekstraklasa, I Liga, ... A-Klasa
Krok 2	Wybór województwa (tylko dla lig regionalnych: IV liga i niżej)
Krok 3	Wybór grupy (np. dla III ligi: Grupa I–IV; dla okręgówki: grupy regionalne)
Krok 4	Wyświetlenie tabeli i meczów wybranej ligi

POMIJANIE KROKÓW

Jeśli szczebel krajowy ma tylko jedną grupę (np. Ekstraklasa), wizard przeskakuje od razu do kroku 4. Jeśli województwo ma tylko jedną grupę danego szczebla – podobnie.

Ulubiona drużyna

Użytkownik może kliknąć gwiazdkę ★ przy dowolnej drużynie. Klub zapisuje się w `localStorage` pod kluczem `meczomat_fav_obj`. Przy każdym wejściu na stronę aplikacja automatycznie ładuje następny mecz tej drużyny.

Globalna wyszukiwarka drużyn

Po załadowaniu aplikacja w tle pobiera dane **wszystkich lig** i buduje słownik wszystkich drużyn (`globalTeams`). Dzięki temu wyszukiwarka na stronie głównej działa globalnie – wpisując nazwę klubu możesz znaleźć go w dowolnej lidze.

WYDAJNOŚĆ

Globalne pobieranie wszystkich lig przy starcie może generować wiele równoległych requestów do API. Jeśli serwer ma cold start (np. po uśpieniu na Render.com), pierwsze ładowanie strony może trwać 10–30 sekund.

App.jsx – główny komponent

Serce aplikacji. Zarządza globalnym stanem, nawigacją i komunikacją między komponentami. Zawiera:

- Stałą LEAGUE_STRUCTURE – hierarchiczny słownik wszystkich lig dostępnych w serwisie
- Stałą WIZARD_LEVELS – lista szczebli rozgrywkowych do kreatora
- Logikę useEffect ładującą: ulubioną drużynę z localStorage, najnowsze treści CMS, wszystkie drużyny globalnie
- Render warunkowy widoków (home / leagues / news / cms)
- Komponent nawigacji (sticky navbar)

LeagueTable.jsx – tabela ligowa

Prosty komponent przyjmujący prop leagueId. Pobiera tabelę z /api/tabela?liga={leagueId} i renderuje ją w stylowej tabeli HTML. Obsługuje:

- Stan ładowania (spinner)
- Podświetlanie ulubionej drużyny
- Responsywne ukrywanie kolumn na mobile (hidden md:table-cell)
- Klikanie w gwiazdkę ★ do zapisywania ulubionego klubu

MatchList.jsx – lista meczów

Pobiera mecze z /api/mecze?liga={leagueId}. Wyświetla rozgrywki pogrupowane po kolejkach. Rozróżnia mecze zakończone (pokazuje wynik) i zaplanowane (pokazuje datę i godzinę).

TeamSearch.jsx – wyszukiwarka i profil drużyny

Komponent z podwójną funkcją:

- **Wyszukiwarka** – filtruje listę globalTeams po wpisanej frazie; kliknięcie przełącza aktywną ligę i otwiera profil drużyny
- **Profil drużyny** – po wyborze drużyny pokazuje jej 5 ostatnich wyników (Z/R/P) i pełen terminarz meczów w danej lidze

CmsPanel.jsx – panel redakcyjny

Panel dla zalogowanych dziennikarzy. Opisany szczegółowo w rozdziale 8.

PublicNews.jsx – Centrum Kibica

Publiczny widok treści z CMS. Trzy zakładki: Wiadomości, Skróty Wideo, Transmisje. Transmisje wyróżnione czerwonym wskaźnikiem LIVE z animacją pulsowania. Embeduje filmy bezpośrednio przez iframe YouTube.

Stylowanie – Tailwind CSS + klasy własne

Projekt używa customowego motywu ciemnego (dark mode). W plikach CSS zdefiniowane są klasy pomocnicze:

KLASA CSS	OPIS
<code>glass-card</code>	Szklany efekt karty z przezroczystością i rozmyciem tła
<code>glass-nav</code>	Wersja dla paska nawigacji (sticky)
<code>glass-surface</code>	Lekka szklana powierzchnia wewnątrz kart
<code>brand-accent</code>	Kolor akcentu – jaskrawa zieleń <code>#00ff88</code>
<code>brand-cream</code>	Główny kolor tekstu – kremowy biały
<code>animate-fade-in</code>	Płynne pojawienie się elementu
<code>animate-slide-up</code>	Wjazd elementu od dołu

7

API – lista wszystkich endpointów

Serwer nasłuchuje pod adresem produkcyjnym `https://meczomat-api.onrender.com`. Lokalnie: `http://localhost:3001`.

Endpointy danych sportowych

GET `/api/tabela?liga={ligaId}`

Zwraca tablicę obiektów tabeli ligowej dla podanej ligi. Odświeża cache co 15 min ze scrapingu 90minut.pl.

GET `/api/mecze?liga={ligaId}`

Zwraca tablicę meczów (zakończonych i zaplanowanych) z terminarzem i wynikami dla podanej ligi.

Endpointy autoryzacji

POST `/api/login`

Logowanie do CMS. Body: `{ username, password }`. Zwraca `{ success: true, username }` lub 401.

Endpointy artykułów

GET `/api/articles`

Pobiera wszystkie artykuły z bazy MongoDB, posortowane od najnowszego.

POST `/api/articles`

Dodaje nowy artykuł. Body: `{ title, content, author, password }`. Wymaga prawidłowego loginu/hasła z JOURNALISTS.

POST `/api/articles/delete`

Usuwa artykuł po ID. Body: `{ id, author, password }`. Wymaga uprawnień.

Endpointy wideo i transmisji (analogiczna struktura)

ENDPOINT	METODA	OPIS
<code>/api/videos</code>	GET	Pobierz wszystkie wideo
<code>/api/videos</code>	POST	Dodaj wideo (wymaga loginu)

<code>/api/videos/delete</code>	POST	Usuń wideo (wymaga loginu)
<code>/api/streams</code>	GET	Pobierz wszystkie transmisje
<code>/api/streams</code>	POST	Dodaj transmisję (wymaga loginu)
<code>/api/streams/delete</code>	POST	Usuń transmisję (wymaga loginu)

FORMAT ODPOWIEDZI

Wszystkie endpointy zwracają JSON. Błędy zwracają odpowiedni kod HTTP (400, 401, 403, 500) z obiektem { error: "..."}.

CMS (Content Management System) to wbudowany panel administracyjny dostępny z menu „ CMS”. Umożliwia dziennikarzom/redaktorom publikowanie treści bez znajomości programowania.

Logowanie

Panel wymaga podania loginu i hasła. Dane są sprawdzane na serwerze przez endpoint `/api/login`. Po zalogowaniu credentials zapisywane są w `localStorage` – użytkownik pozostaje zalogowany po odświeżeniu strony.

Dostępne konta (produkcja)

LOGIN	HASŁO	PRZEZNACZENIE
admin	haslo123	Administrator
redaktor	piłka2025	Redaktor artykułów
kamera	wideo123	Operator wideo

Typy treści w CMS



Artykuły

Tytuł + treść (długi tekst). Pojawia się w zakładce „Wiadomości” w Centrum Kibica. Wyświetlany z formatowaniem white-space: pre-wrap (respektuje Enter).



Skróty Wideo

Tytuł + link YouTube. Panel automatycznie przetwarza link na embed URL. Wyświetlane jako osadzone wideo w zakładce „Skróty Wideo”.



Transmisje Live

Tytuł + link YouTube Live. Identyczne z wideo, ale wyświetlane w osobnej zakładce „Transmisje” z czerwonym wskaźnikiem LIVE.



Usuwanie treści

Każdy zalogowany redaktor może usunąć dowolną treść (artykuł, wideo, stream) przyciskiem kosza w panelu CMS.

Jak opublikować artykuł – krok po kroku

Kliknij przycisk CMS w górnym menu.

2 Zaloguj się

Podaj login i hasło z tabeli powyżej.

3 Wybierz zakładkę

Artykuły / Skróty Wideo / Transmisje – w zależności od typu treści.

4 Wypełnij formularz i kliknij Opublikuj

Treść natychmiast pojawi się w Centrum Kibica dla wszystkich użytkowników.

Wymagania wstępne

- Node.js w wersji 18+ (zalecane 20 LTS)
- npm w wersji 9+
- Dostęp do internetu (scraping 90minut.pl + MongoDB Atlas)

Uruchomienie backendu

```
# W katalogu głównym projektu (meczomat_v2.0/)
npm install
node server.js

# Serwer startuje na http://localhost:3001
# Oczekiwany log:  Serwer działa na http://localhost:3001
# Oczekiwany log:  Sukces! Połączono z chmurową bazą MongoDB!
```

Uruchomienie frontendu

```
# W katalogu meczomat2.0/
cd meczomat2.0
npm install
npm run dev

# Frontend startuje na http://localhost:5173
```

UWAGA – ADRES API W KODZIE FRONTENDU

Frontend jest na razie skonfigurowany tak, że

ZAWSZE

komunikuje się z produkcyjnym API (<https://meczomat-api.onrender.com>), nawet lokalnie. Jeśli chcesz testować z lokalnym backendem, musisz zmienić adresy URL we wszystkich komponentach React na `http://localhost:3001`.

Polecenia npm – podsumowanie

KATALOG	KOMENDA	OPIS
root	<code>node server.js</code>	Uruchamia serwer API na porcie 3001

meczomat2.0/	<code>npm run dev</code>	Dev server frontendu z hot reload (port 5173)
meczomat2.0/	<code>npm run build</code>	Buduje statyczny bundle do folderu <code>dist/</code>
meczomat2.0/	<code>npm run preview</code>	Podgląd zbudowanego bundle'u lokalnie
meczomat2.0/	<code>npm run lint</code>	Uruchamia ESLint na kodzie frontendu

Backend – Render.com

Serwer Node.js jest wdrożony na platformie **Render.com** jako Web Service.

PARAMETR	WARTOŚĆ
Adres produkcyjny	<code>https://meczomat-api.onrender.com</code>
Runtime	Node.js
Start command	<code>node server.js</code>
Plan	Free (uśpienie po 15 min bezczynności)

COLD START

Serwer na darmowym planie Render.com zasypia po 15 minutach braku ruchu. Pierwsze żądanie po uśpieniu powoduje cold start trwający 30–60 sekund. Użytkownicy mogą widzieć ładowanie przy pierwszym wejściu.

Frontend – Netlify / Vercel

Zbudowany bundle React (folder `dist/`) można hostować na dowolnym hostingu statycznym. Zalecaną opcją jest Netlify lub Vercel z automatycznym deploymentem z repozytorium Git.

1

Build frontendu

W katalogu `meczomat2.0/` uruchom `npm run build` → powstaje folder `dist/`.

2

Upload do Netlify

Przeciągnij folder `dist/` na panel Netlify lub podłącz repozytorium Git z build command `npm run build` i publish directory `meczomat2.0/dist`.

3

Konfiguracja SPA routing

Dodaj plik `meczomat2.0/public/_redirects` z zawartością: `/* /index.html 200` (dla Netlify), lub `vercel.json` z rewrites dla Vercel.

Zmienne środowiskowe (TODO)

Rekomendowane przeniesienie wrażliwych danych z kodu do zmiennych środowiskowych na Render.com:

ZMIENNA	WARTOŚĆ	GDZIE USTAWIĆ
<code>MONGO_URI</code>	String połączenia MongoDB Atlas	Render.com → Environment

Ograniczenia obecnego rozwiązania

PROBLEM	PRIORYTET	OPIS
Hasła plain text	Wysoki	Hasła redaktorów i MongoDB URI są wpisane wprost w kodzie. Należy użyć <code>.env</code> i <code>bcrypt</code> .
Brak JWT / sesji	Wysoki	Autoryzacja CMS opiera się na porównaniu hasła per request. Brak tokenów sesji.
Cache tylko w RAM	Średni	Po restarcie serwera cache jest pusty. Warto rozważyć Redis do trwałego cache'u.
Hardcodowane URL API	Średni	Frontend używa produkcyjnego URL nawet lokalnie. Należy dodać plik <code>.env</code> z <code>VITE_API_URL</code> .
Brak obsługi błędów 90minut	Niski	Jeśli 90minut.pl zmieni strukturę HTML, scraping przestanie działać bez żadnych alertów.
Ligi „do uzupełnienia”	Niski	A-Klasa i IV liga opolska mają placeholder <code>TUTAJ_WKLEJ_LINK_Z_90MINUT</code> .
Brak React Router	Niski	Nawigacja przez stan React – brak deep linking (URL nie zmienia się przy przełączaniu widoków).

Sugestie rozwoju

- Przenieść MONGO_URI i hasła do zmiennych środowiskowych (`.env`)
- Wdrożyć JWT do autoryzacji CMS
- Dodać React Router dla lepszego SEO i deep linkowania
- Dodać monitoring scrapingu (alerty gdy tabela jest pusta)
- Rozważyć SSR (Next.js) dla lepszego SEO wyników ligowych
- Uzupełnić brakujące ligi (A-Klasa, IV liga opolska)